1.Aim: To write a C program that simulates a Deterministic Finite Automaton (DFA) to check if a given input string is accepted or rejected by the machine.

Theory: A DFA is a 5-tuple consisting of states, input symbols, transition functions, a start state, and final states. For every state and input symbol, there is exactly one transition to a next state. The program uses a state transition table or a switch-case block to track the current state as characters are read.

A DFA is defined as **$M = (Q, \Sigma, \delta, q_0, F)$**.

The DFA starts in the **initial state q_0** and reads the input string symbol by symbol.

State transitions occur according to the transition function **δ**.

Source Code :

```
#include <stdio.h>

#include <string.h>

int main() {

   char str[100];

   int state = 0;

   printf("Enter string: ");

   scanf("%s", str);

   for (int i = 0; i < strlen(str); i++) {

      if (state == 0 && str[i] == '0') state = 1;

      else if (state == 1 && str[i] == '1') state = 2;

      else state = 0;

   }

   if (state == 2)

      printf("String Accepted\n");

   else

      printf("String Rejected\n");
```

```
    return 0;
}
```

OUTPUT :

Output Clear

```
DFA for strings ending with '1' (Alphabet: 0, 1)
Enter string: 0101
Result: String Accepted


=== Code Execution Successful ===
```

2.Aim: To implement a program that checks if an input string matches specific regular expressions.

Theory: Regular expressions define a search pattern for strings.

- **a.** `10 + (0 + 11)0*1`: Matches "10" OR a "0" or "11" followed by any number of "0"s and ending in "1".
- **b.** `0*1 + 10`: Matches any number of "0"s followed by a "1" OR specifically "10".
- **c.** `(a+b)* c (a+b)*`: Matches any string of 'a's and 'b's that contains at least one 'c'.

Source Code :

a)

```
#include <stdio.h>

#include <string.h>

int main() {

   char s[100];

   printf("Enter string: ");

   scanf("%s", s);

   if (!strcmp(s,"10") || s[0]=='0' && s[strlen(s)-1]=='1')

      printf("Accepted\n");

   else

      printf("Rejected\n");

   return 0;

}
```

OUTPUT :

```
Output                                    Clear
Enter string: 001
Accepted


=== Code Execution Successful ===
```

b)

```
#include <stdio.h>

#include <string.h>

int main() {

   char s[100];

   printf("Enter string: ");

   scanf("%s", s);

   if (strcmp(s,"10")==0 || s[strlen(s)-1]=='1')

      printf("Accepted\n");

   else

      printf("Rejected\n");

   return 0;

}
```

OUTPUT :

Output Clear

```
Enter string: 0001
Accepted


=== Code Execution Successful ===
```

C)

```
#include <stdio.h>

#include <string.h>

int main() {

   char s[100];

   printf("Enter string: ");

   scanf("%s", s);

   if (strchr(s,'c'))

      printf("Accepted\n");

   else

      printf("Rejected\n");

   return 0;

}
```

OUTPUT :

```
Output                                    Clea
Enter string: ababcba


Accepted


=== Code Execution Successful ===
```

3.Aim: To implement a Symbol Table using an array or hash table to perform CRUD operations.

Theory: A symbol table is a data structure used by a compiler to store information about the scope and binding information of names (variables, functions, etc.).

- **Insertion** operation adds a new symbol and its attributes into the table.
- **Display** operation shows all the symbols present in the table.
- **Search** operation checks whether a given symbol exists in the table.
- **Deletion** operation removes a symbol entry from the table.
- **Modification** operation updates the attributes of an existing symbol.

Source Code (C):

```
#include <stdio.h>

#include <string.h>

struct symbol {

   char name[20];

   int addr;

} sym[20];

int count = 0;

void insert() {

   printf("Enter symbol name: ");

   scanf("%s", sym[count].name);

   sym[count].addr = count * 10;

   count++;

}

void display() {

   printf("Symbol\tAddress\n");
```

```
    for(int i=0;i<count;i++)
        printf("%s\t%d\n", sym[i].name, sym[i].addr);
}
int main() {
    int ch;
    while(1) {
        printf("\n1.Insert 2.Display 3.Exit\n");
        scanf("%d",&ch);
        if(ch==1) insert();
        else if(ch==2) display();
        else break;
    }
    return 0;
}
```

OUTPUT :

```
Output                                                    Clear

1.Insert 2.Display 3.Exit
Choice: 1
Enter Name, Type, Address: x int 0

1.Insert 2.Display 3.Exit
Choice: 1
Enter Name, Type, Address: y long 100

1.Insert 2.Display 3.Exit
Choice: 2

Name     Type     Address
x        int      0
y        long         100

1.Insert 2.Display 3.Exit
```

4.Aim: To develop a Lexical Analyser in C that categorizes source code into tokens like keywords, identifiers, and operators.

Theory: Lexical analysis is the first phase of a compiler. It scans the source code as a stream of characters and groups them into meaningful sequences called lexemes, which are then identified as tokens.

Source Code (C):

```
#include <stdio.h>

#include <ctype.h>

int main() {

   char ch;

   printf("Enter expression: ");

   while((ch=getchar())!='\n') {

      if(isalpha(ch))

         printf("%c is Identifier\n",ch);

      else if(isdigit(ch))

         printf("%c is Constant\n",ch);

      else if(ch=='+'||ch=='-'||ch=='*'||ch=='/')

         printf("%c is Operator\n",ch);

   }

   return 0;

}
```

OUTPUT :

```
Output                                    Clear
Enter expression: a+b*45
a is Identifier
+ is Operator
b is Identifier
* is Operator
4 is Constant
5 is Constant


=== Code Execution Successful ===
```

5.Aim: To implement a lexical analyser using the LEX (Flex) tool.

Theory: LEX is a computer program that generates lexical analyzers. It uses regular expressions to describe patterns and generates C code to recognize them. It uses **regular expressions** to specify patterns for tokens such as identifiers, constants, operators, and comments.

The input to LEX is a specification file containing **definitions, rules, and user code**.

LEX generates a C program (`lex.yy.c`) that performs lexical analysis.

The generated analyzer scans the source code and recognizes valid tokens.

Comments and whitespace are ignored, while invalid patterns are reported as errors.

The recognized tokens are passed to the **parser** for further processing.

>A LEX program consists of three sections:

1) **Defination Section:**
   Contains macro defintions and header files.

2) **Rules section:**
   Contains patters (regular expressions) and corresponding actions.

3) **User Code Section:**
   Contains additional C functions like main( ).

Source Code (LEX/Flex):

```
%{

#include <stdio.h>

%}



%%

[0-9]+        { printf("%s is a Constant\n", yytext); }

[a-zA-Z_][a-zA-Z0-9_]* { printf("%s is an Identifier\n", yytext); }

"+"|"-"|"*"|"/" { printf("%s is an Operator\n", yytext); }
```

```
.           ;

%%

int main() {

   yylex();

   return 0;

}

int yywrap() {

            return 1;

}
```

Commands : lex lexer.l -> generates lex.yy.c

gcc lex.yy.c –o lexer

./lexer

OUTPUT :

Output Clear

```
count + 20
count is an Identifier
a is an Identifier
+ is an Operator
* is an Operator
20 is a Constant



=== Code Execution Successful ===
```